Spam Shield

An Project report Submitted in partial fulfillment of the requirements for the award of degree of

BACHELOR OF TECHNOLOGY

in

Computer Science & Engineering

by

R.VisalyaManogna

2200031033

Under the esteemed guidance of

Mr.Sarath Babu
HR
Marketing, Gaint clout technologies pvt ltd

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

K L UNIVERSITY

Green Fields, Vaddeswaram, Tadepalli,
Guntur - 522 502, Andhra Pradesh.

2025-26

Gaint clout technologies pvt ltd

Orbit Knowledge City, Raidurg

Hyderabad, Telangana 500019

Bonafide Certificate

This is to certify that this project report entitled **"SpamShield"** submitted to **Department of Computer Science & Engineering, KL Deeemed to be University, Guntur,** in connection with the University internship program is a bonafide record of work done by "**R.VisalyaManogna"** under my supervision at the "Gaint clout technologies pvt ltd" from 14-07-2025 to 08-10-2025.

Mr.Sarath Babu

HR

K L Deemed to be UNIVERSITY

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Certificate

This is Certified that the project entitled **Spam Sheild** which is a experimental &/ theoretical &/ Simulation&/ hardware work carried out by RAYANKULA VISALYAMANOGNA 2200031033, in partial fulfillment for the award of the degree of **Bachelor of Technology** in Department **COMPUTER SCIENCE & ENGINEERING**, during the year **2025-26**. The project has been approved as it satisfies the academic requirements.

Department Coordinator **Head of the Department**

University Guide **External**

ACKNOWLEDGEMENT

It is with great pleasure that I express my profound gratitude to our Honorable President, **Sri Koneru Satyanarayana**, for providing the opportunity and necessary facilities that enabled me to successfully complete this internship and project report.

I convey my sincere thanks to **Dr.B.Tirupati Reddy**, Head of the Department of Computer Science & Engineering, for his continuous support, motivation, and valuable guidance throughout the course of my academic journey. His leadership has been a constant source of inspiration.

I extend my heartfelt appreciation to my **University Guide, Dr. Venkateswar Rao**, for his insightful suggestions, encouragement, and supervision, which greatly contributed to the successful completion of my project. His guidance helped me bridge the gap between theoretical learning and practical application.

I would also like to express my deep sense of gratitude to my **Company Guide, Mr.Sharath babu**, for his valuable mentorship, expert advice, and constant support during the internship period. His professional experience and guidance provided me with meaningful exposure to industry practices and enriched my learning experience.

I am thankful to the **Regional Manager**, the **Director Of Practice School**, the **Practice School In-Charge**, and all **Department Coordinators** for their assistance and coordination in facilitating the smooth execution of this internship program.

Finally, I wish to acknowledge my gratitude to all faculty members, colleagues, and well-wishers who directly or indirectly contributed to the successful completion of this project report.

Spam Shield

Rayankula VisalyaManogna - 2200031033

Jalamuri Sreeja – 2200030327

R. Anusri - 2200031526

ABSTRACT

In the modern digital communication environment, email remains one of the primary methods of exchanging information. However, the rising prevalence of spam emails poses significant challenges, such as security threats, misinformation, reduced productivity, and increased operational costs. Spam emails often contain phishing links, malicious attachments, or irrelevant content, which can lead to data breaches, financial loss, and degraded user experience.

This project presents Spam Shield, a robust and scalable spam email detection system designed to identify and filter spam emails efficiently. The application leverages machine learning algorithms and natural language processing techniques to analyze incoming emails and classify them as spam or legitimate. The frontend is developed using Node.js and React, providing users with a seamless interface for viewing, filtering, and managing emails.

The system architecture integrates various modules such as email parsing, feature extraction, classification using supervised learning models, and real-time spam detection. The application ensures privacy and security by processing emails locally or on secure servers, and by minimizing exposure of sensitive information.

Spam Shield is intended for organizations and individuals who seek to enhance their email security, reduce spam-related distractions, and improve productivity. The solution emphasizes accuracy, ease of use, and adaptability to evolving spam patterns. This report covers the problem statement, existing methods, proposed architecture, implementation details, dataset description, evaluation metrics, and future improvements..

TABLE OF CONTENTS

S.NO	TITLE	PAGE NO
1	Introduction	9 – 12
2	Literature Review	13 – 15
3	Modules And Feature	16 - 17
4	Implementation and file structure	18 - 46
5	Discussion	47
6	Conclusion & Future Work	48
7	References	49

LIST OF TABLES

3.1 Input Modules

3.2 Feature Categories in Each Module

LIST OF FIGURES

4.4 Output of Signup Page

4.5 Output of Login Page

4.6 Output of Dashboard Page

4.7 Output of Email Actions

4.8 Output of Settings Page

List of Symbols, Abbreviations, and Nomenclature

AI - Artificial Intelligence

ML - Machine Learning

NLP - Natural Language Processing

UI - User Interface

API - Application Programming Interface

DB - Database

MVP - Minimum Viable Product

SMTP - Simple Mail Transfer Protocol

IMAP - Internet Message Access Protocol

JWT - JSON Web Token

AES-256 - Advanced Encryption Standard(256-bit)

SVM - Support Vector Machine

KNN - K-Nearest Neighbors

CHAPTER-1

INTRODUCTION

1.1 Background and Motivation

Email communication is essential in both personal and professional settings, but spam emails have become a major problem. These unwanted messages often contain phishing links, malware, and irrelevant content that threaten productivity and security. Traditional filtering methods, like keyword-based rules or simple blacklists, are not effective against constantly changing spam tactics, resulting in blocked legitimate emails or missed spam.

With advancements in Artificial Intelligence (AI) and Machine Learning (ML), smarter spam detection systems can now analyze email content and metadata to classify messages more accurately. Techniques such as Natural Language Processing (NLP), TF-IDF, and algorithms like Support Vector Machine (SVM) and Naïve Bayes help distinguish spam from legitimate emails. The Spam Shield project uses these technologies to create a real-time spam filtering platform with a user-friendly interface built using Node.js and React.

As reliance on email increases, spam threats continue to grow, making automated, adaptive solutions necessary. Spam Shield aims to enhance communication security by reducing spam disruptions, improving email management, and safeguarding user data. The project's goal is to provide an efficient and trustworthy tool that supports safe and uninterrupted email communication in today's digital environment.

1.2 Problem Statement

Despite the widespread use of email communication, there remains a significant gap in the availability of reliable, adaptive, and secure spam detection tools. Existing email filtering solutions face several challenges that impact their effectiveness and user experience:

1. Inefficient Filtering: Traditional spam filters based on keywords or static rules often fail to detect new spam techniques, allowing harmful emails to bypass protection or incorrectly block legitimate messages..
2. High False Positives and Negatives: Many spam detection systems either flag too many genuine emails as spam or fail to identify spam emails, reducing trust in email security solutions.
3. Limited Adaptability: Current systems struggle to adjust to rapidly changing spam patterns and emerging threats, making them less effective over time without continuous manual updates.
4. Poor User Experience: Some solutions require complex configurations or lack intuitive interfaces, making it difficult for users to manage their emails efficiently.
5. Privacy and Security Risks: Email content and metadata are sensitive information; inadequate handling of this data may lead to privacy breaches or unauthorized access.

Therefore, there is a pressing need for an AI-driven, adaptive, and privacy-conscious spam detection system that can classify emails accurately in real time. The solution should be scalable, easy to use, and capable of evolving with new spam tactics while protecting user data. Spam Shield aims to address these challenges by providing a robust and intelligent platform that enhances email security and user confidence in communication systems.

1.3 Objectives

The primary objective of the Spam Shield project is to design and implement an AI-powered spam email detection system that is accurate, adaptive, and user-friendly while ensuring data security and privacy.

The specific objectives are as follows:

- Develop robust modules using ML and NLP to analyze email content, metadata, and structure for accurate spam detection and adaptability to new patterns.
- Design a database to store email patterns, user preferences, and classification results with encryption and controlled access.
- Provide instant feedback on email status with clear indicators and confidence levels for effective inbox management.
- Implement encryption and secure token-based authentication, prioritizing user consent to ensure trust and compliance.
- Use Node.js and React to create a user-friendly platform for easy email management and filter customization.
- Protect users in personal, business, and online communication from phishing, malware, and other spam threats.
- To follow a phased roadmap, starting with an MVP and gradually adding features and improvements.

1.4 Scope and Limitations

Scope:

- The Spam Detection System will be available as a web application for personal and business email accounts.
- The system will analyze email content, metadata, and structure in real-time for accurate spam detection.
- It will provide features such as spam classification, confidence indicators, user-defined filters, and reporting.
- Target users include individual email users, organizations, and IT administrators seeking enhanced email security.
- The platform will support secure user registration, preference management, and token-based authentication.
- The system will leverage cloud infrastructure for scalability and offer API endpoints for integration with third-party email services.

Limitations:

- The system is not intended to replace enterprise-grade email security solutions or legal investigations.
- Detection accuracy may be affected by highly sophisticated or obfuscated spam emails.
- The system provides probabilistic spam predictions; false positives and negatives are possible.
- Ethical use requires user consent and privacy compliance; misuse for unauthorized monitoring is prohibited.
- Continuous updates to algorithms and datasets are required to handle evolving spam patterns and minimize biases.

CHAPTER-2

LITERATURE REVIEW

2.1 Existing Tools and Methods

2.1.1 Manual Email Filtering

Traditional email filtering relies on users manually marking emails as spam or moving them to folders. While simple, this method is error-prone and time-consuming. Users often miss spam emails or incorrectly flag legitimate messages, making it inefficient for large-scale email management.

2.1.2 Rule-Based Spam Filters

Rule-based systems use predefined patterns, keywords, or sender blacklists to identify spam. Although effective for known spam types, these methods struggle with evolving spam tactics, phishing attempts, and sophisticated obfuscation techniques. Maintenance of rules can also be labor-intensive.

2.1.3 Machine Learning-Based Spam Detection

Recent research has focused on machine learning and natural language processing (NLP) for spam detection. Algorithms such as Naive Bayes, SVM, Random Forest, and deep learning models analyze email content, metadata, and structure to classify messages. These approaches improve detection accuracy and adaptability but require large datasets and ongoing updates to handle new spam patterns.

2.1.4 Cloud-Based and Real-Time Systems

Modern solutions leverage cloud infrastructure to process emails in real-time, providing instant classification and user notifications. These systems integrate APIs for third-party email services and can scale for enterprise-level usage. Challenges include ensuring data privacy, secure token management, and minimizing false positives.

2.2 Gaps in Current Systems

Despite advances in email filtering and spam detection, current systems have several limitations:

1. Limited Adaptability to New Spam: Many rule-based or keyword-based filters fail to detect evolving spam patterns, phishing attempts, and obfuscated messages.
2. False Positives and Negatives: Existing systems often misclassify legitimate emails as spam or miss actual spam, reducing user trust and efficiency.
3. Scalability Issues: Manual or rule-based approaches struggle to handle large volumes of email in real-time, especially for enterprise-level users.
    a. Privacy and Security Concerns: Some cloud-based spam detection tools may lack robust encryption or secure token management, risking exposure of sensitive email data.
4. Lack of Explainability: Machine learning-based systems often act as “black boxes,” offering spam/ham classifications without explaining the rationale behind decisions, making it harder for users to trust the system.

2.3 Research Relevance to the Spam Detection System

The Spam Detection System is designed to address the limitations of existing email filtering methods by providing an adaptive, secure, and user-friendly solution. Its contributions over current systems include:

• Enhanced Adaptability: Uses advanced ML and NLP techniques to detect evolving spam patterns and sophisticated phishing attempts.

• Improved Accuracy: Reduces false positives and negatives through probabilistic models and confidence scoring.

• Real-Time Processing: Provides instant spam classification and notifications for efficient inbox management.

• Data Privacy and Security: Implements encryption, secure token-based authentication, and consent-driven user controls.

• User-Centric Customization: Allows users to define filters, preferences, and thresholds tailored to their inbox needs.

• Explainable Decisions: Offers interpretable results and confidence indicators to help users understand why an email is classified as spam.

.

By addressing these research gaps, the Spam Detection System stands at the intersection of AI and cybersecurity research, contributing to the development of secure, adaptive, and user-friendly email protection technologies.

Chapter 3

MODULES AND FEATURES

The system is divided into three major categories of modules: Input Modules, Processing Modules, and Output Modules. Each module operates independently but integrates seamlessly through the Spam Detection Engine.

3.1 Input Modules

Module Type	Module Name	Description	Output Example
Input	Email Input	Collects emails and metadata.	Email content and metadata
	User Preferences	Stores custom rules, filters, and thresholds.	Filter settings, preferences
	Historical Data	Provides past emails for training and analysis.	Labeled dataset (spam/ham)

Table 3.1: Module Overview

3.2 Processing Modules

- Preprocessing Unit: Cleans and normalizes email text, removes HTML tags, punctuation, and stopwords, and prepares content for feature extraction.
- Feature Extraction Unit: Extracts features from email content, metadata, and structure, including word frequencies, sender patterns, and header analysis.
- Spam Detection Engine: Classifies emails as spam or legitimate using machine learning/NLP models and computes confidence scores.

Module	Features Extracted	Tools/Techniques
Text Analysis	Keywords, phrases, word frequency, sentiment	NLP libraries (NLTK, spaCy)
Metadata Analysis	Sender, domain, headers, IP patterns	Python email, regex
Classification	Spam probability, confidence score	ML models: Naive Bayes, SVM, Random Forest, Deep Learning

Table 3.2: Feature Categories in Each Module

3 Output Modules

- Spam Classification Result: Displays whether an email is spam or legitimate along with a confidence score.
- Confidence Level: Shows the model's probability to help users judge reliability.
- Detailed Report: Maintains logs of analyzed emails, timestamps, sender/receiver info, and classification results.
- Export Functionality: Allows exporting reports in PDF or CSV format for audits, tracking, or further analysis.

Chapter 4

IMPLEMENTATION AND FILE STRUCTURE

4.1 Implementation Overview

The frontend of the Spam Detection System is built with React.js and Tailwind CSS, offering a responsive interface for managing emails and viewing spam detection results. It communicates with the backend API for processing and classification.

Key features implemented in the frontend include:

- User registration, login, and secure authentication
- Dashboard for viewing inbox emails with spam/ham labels
- Real-time display of spam classification results and confidence scores
- Customizable filters and user preferences for email management
- Export and download of email analysis reports in PDF or CSV formats

---

4.2 File Structure

Below is a typical file structure for the frontend React application:

```
SpamShieldX-main/
├── src/
│   ├── app/
│   │   ├── (auth)/
│   │   │   ├── login/
│   │   │   │   └── page.tsx
│   │   │   ├── signup/
│   │   │   │   └── page.tsx
│   │   ├── (main)/
│   │   │   ├── inbox/
│   │   │   │   └── page.tsx
```

```
│   │   │   └── email-actions.ts
│   ├── components/
│   │   ├── app-logo.tsx
│   │   ├── email/
│   │   │   └── email-list.tsx
│   │   ├── ui/
│   │   │   ├── card.tsx
│   │   │   ├── button.tsx
│   │   │   ├── input.tsx
│   │   │   └── label.tsx
│   ├── lib/
│   │   ├── data.ts
│   │   ├── auth.ts
│   ├── hooks/
│   │   └── use-toast.ts
│   ├── ai/
│   │   └── flows/
│   │       └── report-misclassified-email.ts
├── tsconfig.json
├── package.json
```

4.3 Key Files and Directories

- components/: Reusable UI elements like EmailList, EmailItem, Navbar, and Modal
- pages/: Main views: Dashboard/Home, Login/Register, and Reports.
- services/: Handles API calls for spam detection, user authentication, and data retrieva.
- App.js: Sets up routing and global layout.
- index.js: Entry point that renders the app to the DOM.
- styles/: Tailwind CSS and custom styles.

Dashboard.jsx code:

```
"use client"

import Link from 'next/link';

import { usePathname } from 'next/navigation';

import { Inbox, Settings, ShieldAlert, Menu, LogOut, UserCircle } from 'lucide-react';

import { AppLogo } from '@/components/app-logo';

import { Button } from '@/components/ui/button';

import { Sheet, SheetContent, SheetTrigger } from '@/components/ui/sheet';

import { DropdownMenu, DropdownMenuContent, DropdownMenuItem, DropdownMenuLabel,
DropdownMenuSeparator, DropdownMenuTrigger } from '@/components/ui/dropdown-menu';

import { useAuth } from '@/lib/auth';


export default function DashboardLayout({ children }: { children: React.ReactNode }) {

  const pathname = usePathname();

  const { user, logout } = useAuth();

  const menuItems = [

    { href: '/inbox', icon: Inbox, label: 'Inbox' },

    { href: '/quarantine', icon: ShieldAlert, label: 'Quarantine' },

    { href: '/settings', icon: Settings, label: 'Settings' },

  ];

  const NavContent = () => (

    <nav className="grid items-start px-2 text-sm font-medium lg:px-4">

      {menuItems.map((item) => (

        <Link

          key={item.href}

          href={item.href}

          className={`flex items-center gap-3 rounded-lg px-3 py-2 transition-all hover:text-primary ${

         pathname === item.href

    ? 'bg-muted text-primary'

              : 'text-muted-foreground'

          }`}

        >
```

```
      </Link>
    ))}
  </nav>
);
return (
  <div className="grid min-h-screen w-full md:grid-cols-[220px_1fr] lg:grid-cols-[280px_1fr]">
    <div className="hidden border-r bg-card md:block">
      <div className="flex h-full max-h-screen flex-col gap-2">
        <div className="flex h-14 items-center border-b px-4 lg:h-[60px] lg:px-6">
          <AppLogo />
        </div>
        <div className="flex-1">
          <NavContent />
        </div>
      </div>
    </div>
    <div className="flex flex-col">
      <header className="flex h-14 items-center gap-4 border-b bg-card px-4 lg:h-[60px] lg:px-6">
        <Sheet>
          <SheetTrigger asChild>
            <Button
              variant="outline"
              size="icon"
          className="shrink-0 md:hidden"
            >
              <Menu className="h-5 w-5" />
              <span className="sr-only">Toggle navigation menu</span>
            </Button>
          </SheetTrigger>
          <SheetContent side="left" className="flex flex-col">
<div className="flex h-14 items-center border-b px-4 lg:h-[60px] lg:px-6 -mx-4">
              <AppLogo /
```

```
            </SheetContent>
            </Sheet>
            <div className="md:hidden">
              <AppLogo />
            </div>
             <div className="ml-auto">
               <DropdownMenu>
                <DropdownMenuTrigger asChild>
                  <Button variant="ghost" size="icon" className="rounded-full">
                    <UserCircle className="h-6 w-6" />
                    <span className="sr-only">Toggle user menu</span>
                  </Button>
                </DropdownMenuTrigger>
                <DropdownMenuContent align="end">
                  <DropdownMenuLabel>{user?.email}</DropdownMenuLabel>
                  <DropdownMenuSeparator />
                  <DropdownMenuItem onClick={logout}>
                    <LogOut className="mr-2 h-4 w-4" />
                    <span>Logout</span>
                  </DropdownMenuItem>
                </DropdownMenuContent>
               </DropdownMenu>
            </div>
          </header>
          <main className="flex flex-1 flex-col gap-4 overflow-auto bg-background">
             {children}
          </main>
        </div>
      </div>
    );
  }
```

Email-list.tsx code:

```
'use client';
import { useTransition, useState, useEffect } from 'react';
import type { Email } from '@/lib/data';
import { useToast } from '@/hooks/use-toast';
import {
  Accordion,
  AccordionContent,
  AccordionItem,
  AccordionTrigger,
} from '@/components/ui/accordion';
import { Avatar, AvatarFallback, AvatarImage } from '@/components/ui/avatar';
import { Button } from '@/components/ui/button';
import { Card } from '@/components/ui/card';
import { Separator } from '@/components/ui/separator';
import { Inbox, Loader2, ShieldAlert, Trash2, Ban, AlertCircle } from 'lucide-react';
import { cn } from '@/lib/utils';
import { handleReportAction, blockIpAddress } from '@/app/actions/email-actions';

function EmailActions({ email, type }: { email: Email; type: 'inbox' | 'quarantine' }) {
  const [isPending, startTransition] = useTransition();
  const { toast } = useToast();

  const handleReportAsSpam = () => {
    startTransition(async () => {
      const result = await handleReportAction(email.body, true);
      toast({
        title: result.success ? 'Reported as Spam' : 'Error',
        description: result.message,
        variant: result.success ? 'default' : 'destructive',
      });
    });
  };
```

```
  startTransition(async () => {
    const result = await handleReportAction(email.body, false);
    toast({
      title: result.success ? 'Moved to Inbox' : 'Error',
      description: result.message,
      variant: result.success ? 'default' : 'destructive',
    });
  });
};

const handleDelete = () => {
  toast({
      title: 'Email Deleted',
      description: 'The email has been moved to the trash.',
  });
}

const handleBlockIp = () => {
  startTransition(async () => {
      const result = await blockIpAddress(email.ipAddress);
      toast({
          title: result.success ? 'IP Blocked' : 'Error',
          description: result.message,
          variant: result.success ? 'default' : 'destructive',
      });
  });
};
return (
  <div className="flex items-center gap-2">
    {isPending && <Loader2 className="h-4 w-4 animate-spin" />}
    {type === 'inbox' ? (
      <Button variant="outline" size="sm" onClick={handleReportAsSpam} disabled={isPending}>
        <ShieldAlert className="h-4 w-4 mr-2" />
```

```
          ) : (
          <Button variant="default" size="sm" onClick={handleMoveToInbox} disabled={isPending}>
            <Inbox className="h-4 w-4 mr-2" />
            Not Spam
          </Button>
        )}
        <Button variant="outline" size="sm" onClick={handleBlockIp} disabled={isPending}>
          <Ban className="h-4 w-4 mr-2" />
          Block IP
        </Button>
        <Button variant="outline" size="sm" onClick={handleDelete} disabled={isPending}>
          <Trash2 className="h-4 w-4 mr-2" />
          Delete
        </Button>
      </div>
    );
  }
export function EmailList({ emails: initialEmails, type }: { emails: Email[]; type: 'inbox' | 'quarantine'
}) {
  const [emails, setEmails] = useState<Email[]>([]);
  const [isLoading, setIsLoading] = useState(true);
  const [error, setError] = useState<string | null>(null);
  const { toast } = useToast();
  useEffect(() => {
    try {
      // Ensure we have valid emails data
      if (Array.isArray(initialEmails)) {
        setEmails(initialEmails.filter(email => email && typeof email === 'object'));
      } else {
        console.error('Invalid emails data:', initialEmails);
        setError('Failed to load emails. Please try again later.');
      }
    } catch (err) {
```

```
      setError('An error occurred while loading emails.');
    }
finally {
      setIsLoading(false);
    }
  }, [initialEmails]);

  if (isLoading) {
    return (
      <div className="flex flex-col items-center justify-center h-64">
        <Loader2 className="h-8 w-8 animate-spin text-muted-foreground" />
        <p className="mt-2 text-sm text-muted-foreground">Loading emails...</p>
      </div>
    );
  }
  if (error) {
    return (
      <div className="flex flex-col items-center justify-center h-64 p-4 text-center">
        <AlertCircle className="h-8 w-8 text-destructive mb-2" />
        <h3 className="text-lg font-medium">Something went wrong</h3>
        <p className="text-sm text-muted-foreground">{error}</p>
        <Button
          variant="outline"
          className="mt-4"
          onClick={() => window.location.reload()}
        >
          Retry
        </Button>
      </div>
    );
  }
```

```
      <div className="flex flex-col items-center justify-center h-64">
        <Inbox className="h-10 w-10 text-muted-foreground mb-2" />
        <p className="text-sm text-muted-foreground">
          {type === 'inbox' ? 'Your inbox is empty.' : 'No emails in quarantine.'}
        </p>
      </div>
    );
  }
  return (
    <div className="p-4 space-y-2">
      {emails.map((email) => (
        <Card key={email.id} className={cn("transition-all", !email.isRead && type === "inbox" &&
"bg-secondary/40")}>
            <Accordion type="single" collapsible>
                <AccordionItem value={email.id} className="border-b-0">
                    <AccordionTrigger className="p-4 text-left hover:no-underline
[&[data-state=open]]:bg-muted/50">
                        <div className="flex items-start gap-4 w-full">
                            <Avatar className="hidden h-10 w-10 sm:flex">
                                <AvatarImage src={email.avatar} alt={email.from} data-ai-hint="person face" />
                                <AvatarFallback>{email.from.charAt(0)}</AvatarFallback>
                            </Avatar>
                            <div className="grid gap-1 flex-1 text-left">
                                <p className="text-sm font-medium leading-none">{email.from}</p>
                                <p className="text-sm text-muted-foreground truncate">{email.subject}</p>
                            </div>
<div className="ml-auto text-xs text-muted-foreground self-start"> {email.date}


                        </div>
                        </div>
                    </AccordionTrigger>
                    <AccordionContent>
                        <div className="p-4 bg-muted/50 text-sm text-muted-foreground flex
```

```
                <div>IP: {email.ipAddress}</div>
            </div>
            <Separator />
            <div className="p-4 text-sm whitespace-pre-wrap">
                {email.body}
            </div>
            <Separator />
            <div className="p-4 flex justify-end">
                <EmailActions email={email} type={type} />
            </div>
          </AccordionContent>
        </AccordionItem>
      </Accordion>
    </Card>
  ))}
</div>
);
```

INDEX.CSS CODE:

```
import { emails } from '@/lib/data';
import { EmailList } from '@/components/email/email-list';

export default async function InboxPage() {
  const inboxEmails = emails.filter((email) => !email.isSpam);

  return (
    <div className="flex h-full flex-col">
      <header className="border-b bg-card p-4">
        <h1 className="text-2xl font-bold tracking-tight">Inbox</h1>
        <p className="text-sm text-muted-foreground">
          You have {inboxEmails.length} messages in your inbox.
        </p>
      </header>
      <div className="flex-1 overflow-auto">
        <EmailList emails={inboxEmails} type="inbox" />
      </div>
    </div>
  );
```

QUARANTINE CSS CODE:

```
import { emails } from '@/lib/data';
import { EmailList } from '@/components/email/email-list';
import { AlertCircle } from 'lucide-react';
import { Alert, AlertDescription, AlertTitle } from '@/components/ui/alert';

export default async function QuarantinePage() {
  const spamEmails = emails.filter((email) => email.isSpam);

  return (
    <div className="flex h-full flex-col">
      <header className="border-b bg-card p-4">
        <h1 className="text-2xl font-bold tracking-tight">Quarantine</h1>
        <p className="text-sm text-muted-foreground">
          {spamEmails.length} emails flagged as potential spam.
        </p>
      </header>
      <div className="flex-1 overflow-auto">
        <div className="p-4">
            <Alert>
                <AlertCircle className="h-4 w-4" />
                <AlertTitle>Review Required</AlertTitle>
                <AlertDescription>
                    Emails in this folder are suspected to be spam. Review them carefully before deleting. You can move legitimate emails back to your inbox.
                </AlertDescription>
            </Alert>
        </div>
```

```
 <EmailList emails={spamEmails} type="quarantine" />
      </div>
    </div>
  );
}
```

SETTINGS CLIENT PAGE :

```
'use client';
import { useState, useTransition } from 'react';
import { Button } from '@/components/ui/button';
import { Card, CardContent, CardDescription, CardHeader, CardTitle } from '@/components/ui/card';
import { Input } from '@/components/ui/input';
import { Tabs, TabsContent, TabsList, TabsTrigger } from '@/components/ui/tabs';
import { Badge } from '@/components/ui/badge';
import { Trash2, Sparkles, Loader2, PlusCircle, Check } from 'lucide-react';
import { useToast } from '@/hooks/use-toast';

type SettingsClientPageProps = {
    suggestKeywordsAction: () => Promise<string[]>;
    initialKeywords: string[];
    initialIps: string[];
};

function FilterManager({ title, description, items, onAdd, onRemove }: {
    title: string;
    description: string;
    items: string[];
    onAdd: (item: string) => void;
    onRemove: (item: string) => void;
}) {
    const [newItem, setNewItem] = useState('');
```

```
    const handleAdd = (e: React.FormEvent) => {
      e.preventDefault();
      if(newItem.trim()) {
        onAdd(newItem.trim());
        setNewItem('');
      }
    };
    return (
      <Card>
        <CardHeader>
          <CardTitle>{title}</CardTitle>
          <CardDescription>{description}</CardDescription>
        </CardHeader>
        <CardContent>
          <form onSubmit={handleAdd} className="flex items-center gap-2 mb-4">
            <Input
              placeholder={`New ${title.toLowerCase().slice(0,-1)}...`}
              value={newItem}
              onChange={(e) => setNewItem(e.target.value)}
            />
            <Button type="submit"><PlusCircle className="mr-2 h-4 w-4" /> Add</Button>
          </form>
          <div className="flex flex-wrap gap-2">
            {items.length > 0 ? items.map(item => (
              <Badge key={item} variant="secondary" className="animate-in fade-in-0 zoom-in-95
text-sm font-normal pl-3 pr-1 py-1">
                {item}
                <button onClick={() => onRemove(item)} className="ml-2 rounded-full hover:bg-
muted-foreground/20 p-0.5">
                  <Trash2 className="h-3 w-3" />
                  <span className="sr-only">Remove {item}</span>
```

```
 </button>
                    </Badge>
                )) : <p className="text-sm text-muted-foreground">No items in this list.</p>}
            </div>
        </CardContent>
    </Card>
  )
}

export function SettingsClientPage({ suggestKeywordsAction, initialKeywords, initialIps }:
SettingsClientPageProps) {
  const [keywords, setKeywords] = useState<string[]>(initialKeywords);
  const [blacklistedIps, setBlacklistedIps] = useState<string[]>(initialIps);
  const [suggestedKeywords, setSuggestedKeywords] = useState<string[]>([]);

  const [isPending, startTransition] = useTransition();
  const { toast } = useToast();

  const addKeyword = (keyword: string) => {
    if (!keywords.includes(keyword)) {
      setKeywords([...keywords, keyword]);
    }
  };

  const removeKeyword = (keyword: string) => {
    setKeywords(keywords.filter(k => k !== keyword));
  };

  const addIp = (ip: string) => {
    if (!blacklistedIps.includes(ip)) {
      setBlacklistedIps([...blacklistedIps, ip]);
```

```
}
};

const removeIp = (ip: string) => {
  setBlacklistedIps(blacklistedIps.filter(i => i !== ip));
};

const onSuggestKeywords = () => {
  startTransition(async () => {
    const suggestions = await suggestKeywordsAction();
    if (suggestions && suggestions.length > 0) {
      setSuggestedKeywords(suggestions);
    } else {
      toast({
        title: 'AI Suggestions Failed',
        description: 'Could not fetch keyword suggestions at this time.',
        variant: 'destructive',
      });
    }
  });
};

const addSuggestedKeyword = (keyword: string) => {
  addKeyword(keyword);
  setSuggestedKeywords(suggestedKeywords.filter(k => k !== keyword));
}

return (
  <Tabs defaultValue="keywords" className="space-y-4">
    <TabsList>
```

```
  <TabsTrigger value="keywords">Keyword Filters</TabsTrigger>
            <TabsTrigger value="ip-blacklist">IP Blacklist</TabsTrigger>
          </TabsList>
          <TabsContent value="keywords" className="space-y-4">
            <FilterManager
              title="Keywords"
              description="Emails containing these keywords will be marked as spam."
              items={keywords}
              onAdd={addKeyword}
              onRemove={removeKeyword}
            />
            <Card>
              <CardHeader>
                <CardTitle>AI Keyword Suggestions</CardTitle>
                <CardDescription>Let our AI suggest keywords based on common spam
patterns.</CardDescription>
              </CardHeader>
              <CardContent>
                <Button onClick={onSuggestKeywords} disabled={isPending}>
                  {isPending ? <Loader2 className="mr-2 h-4 w-4 animate-spin" /> : <Sparkles
className="mr-2 h-4 w-4" />}
                  Suggest Keywords
                </Button>
                {suggestedKeywords.length > 0 && (
                  <div className="mt-4 space-y-2">
                    <h4 className="font-medium">Click to add a suggestion:</h4>
                    <div className="flex flex-wrap gap-2">
                      {suggestedKeywords.map(keyword => (
                        <Button key={keyword} variant="outline" size="sm" onClick={() =>
addSuggestedKeyword(keyword)} className="animate-in fade-in-0 zoom-in-95">
                          <PlusCircle className="mr-2 h-4 w-4"/>
                          {keyword}
                        </Button>
```

```
                    ))}
                  </div>
                </div>
              )}
            </CardContent>
          </Card>
        </TabsContent>
        <TabsContent value="ip-blacklist">
          <FilterManager
            title="IP Addresses"
            description="Emails from these IP addresses will be blocked."
            items={blacklistedIps}
            onAdd={addIp}
            onRemove={removeIp}
          />
        </TabsContent>
      </Tabs>
    );
  }
```

These are the main codes that are used to implement my project
Outputs are projected below

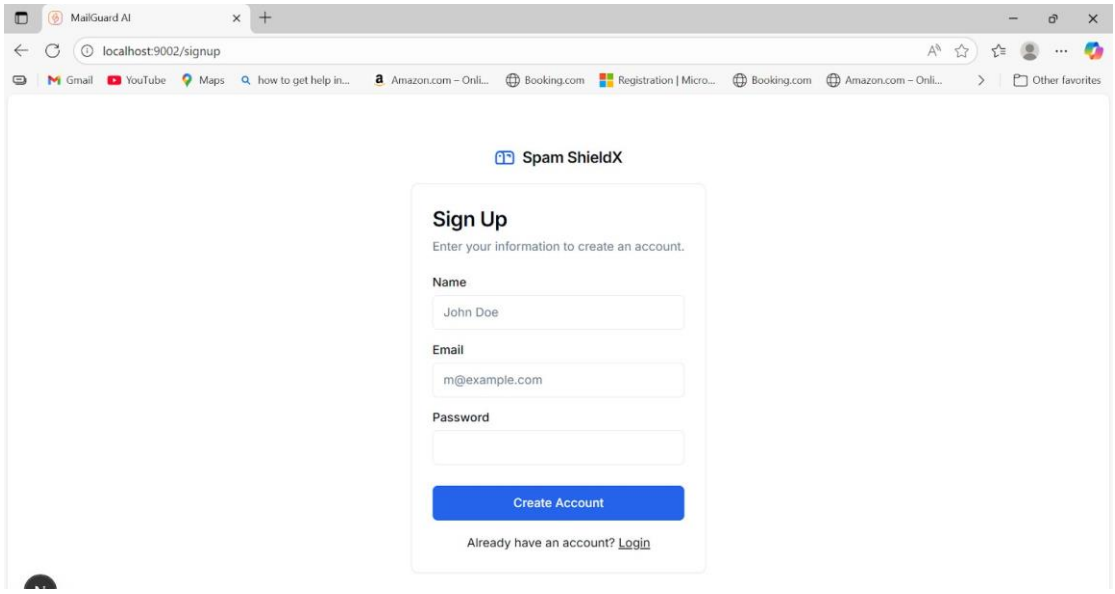

Figure4.4 Output of Signup page

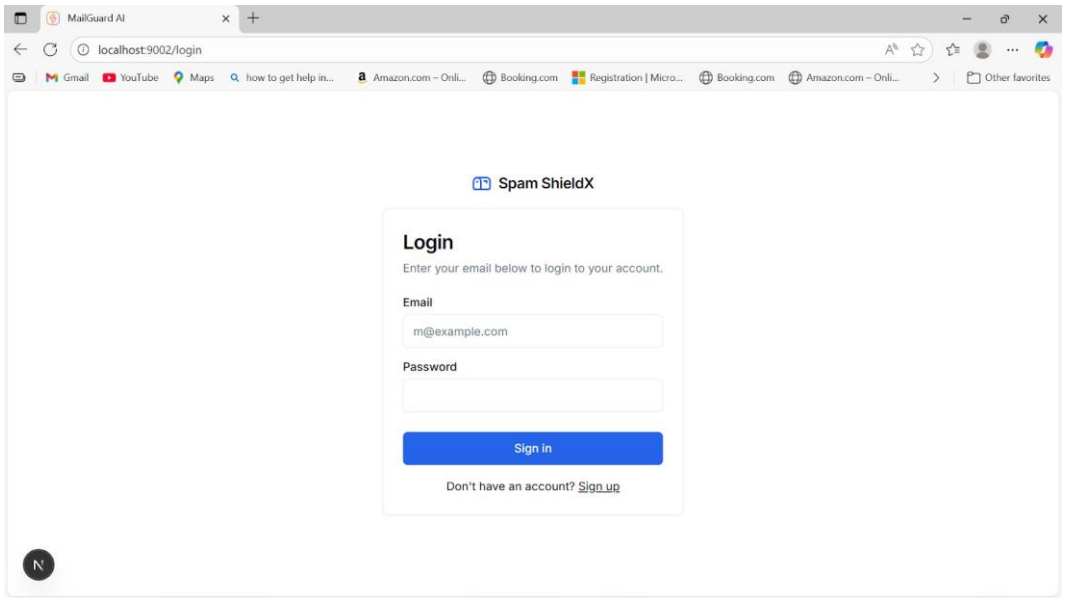

Figure 4.5 Output of login page

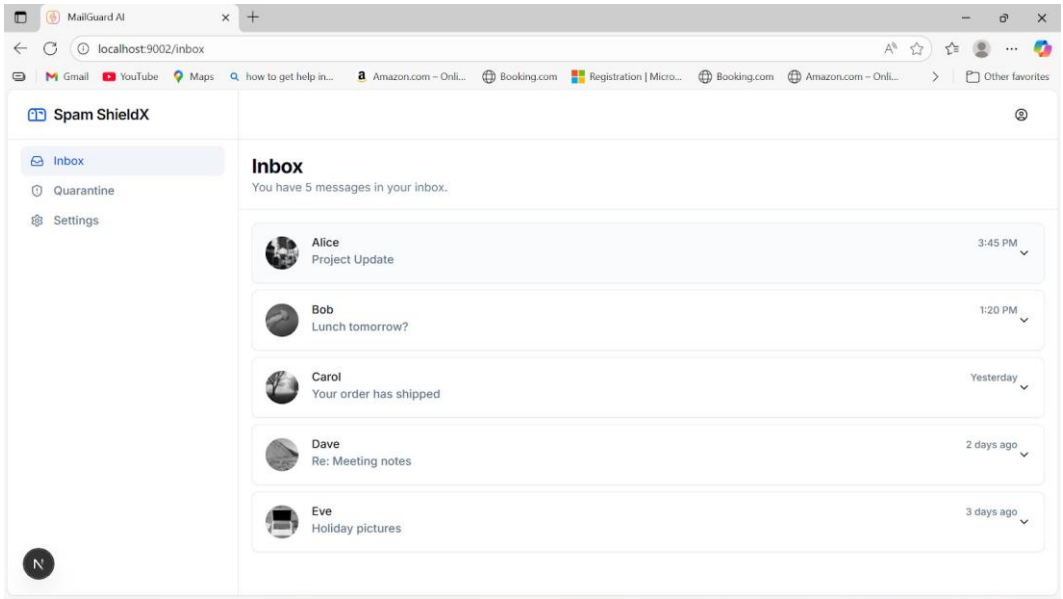

Figure4.6 Output of DashBoard page

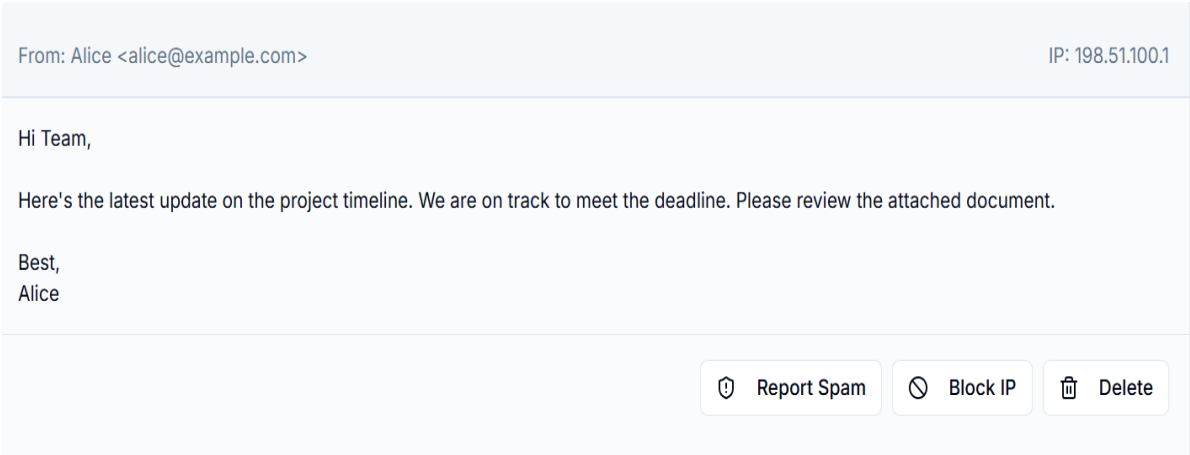

Figure4.7.1 Output of Email Actions

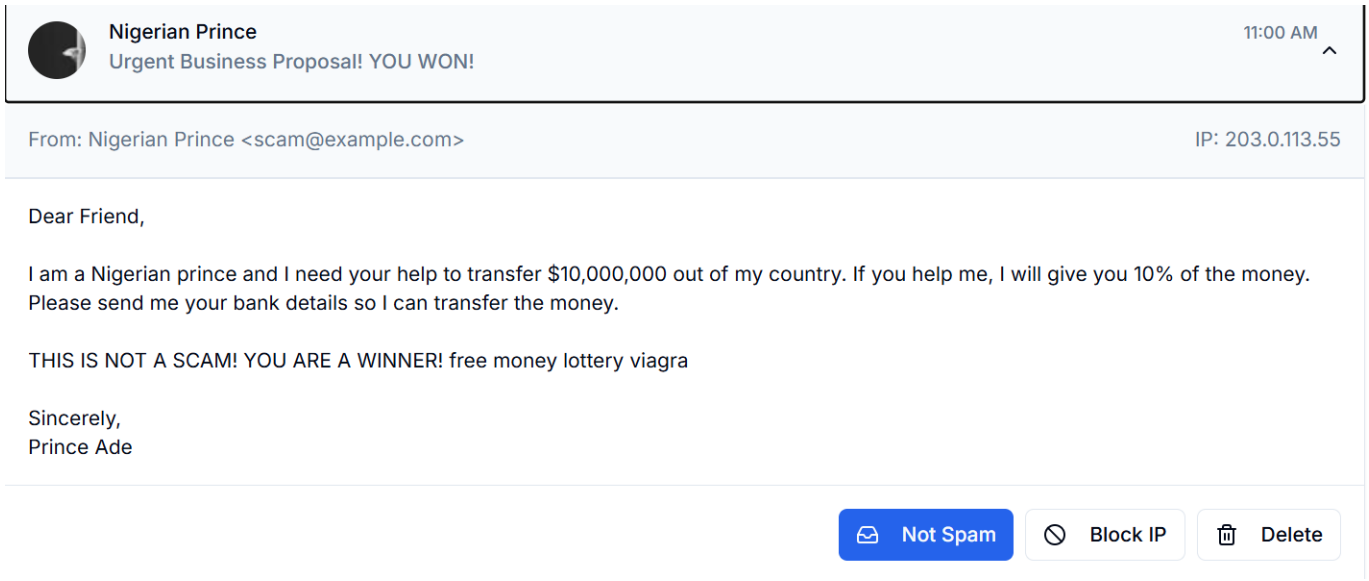

Figure 4.7.2 Output of Email Actions

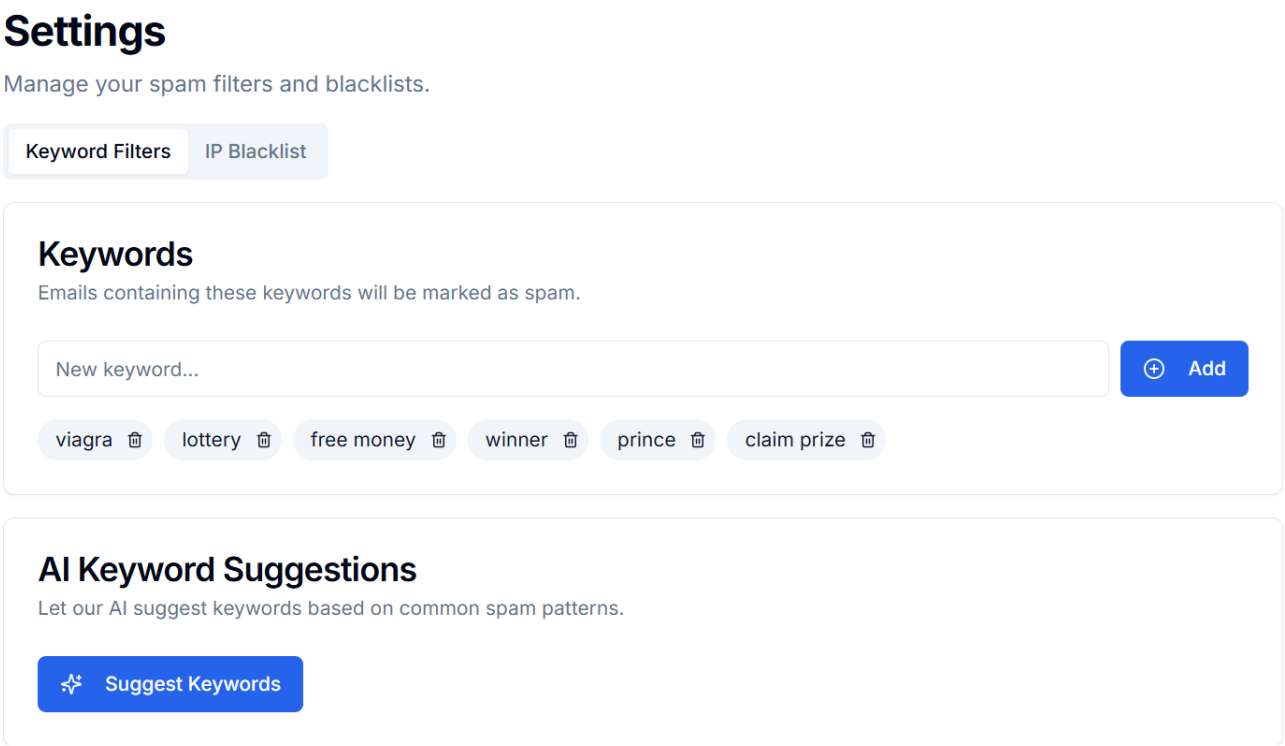

Figure 4.8 Output of Settings page

Chapter 5

DISCUSSION

The Spam Email Detection system processes sensitive user data, including email content, metadata, and sender information, to identify and filter spam messages. While this functionality improves email security and user productivity, it raises important privacy, ethical, and legal considerations. This chapter discusses how the system ensures responsible development and deployment, including secure data handling, user consent, and accurate classification to minimize false positives and maintain trust.

5.1 Privacy Considerations

Consent First: Users must provide explicit consent before their emails and metadata are processed.
Right to Delete: Users can permanently delete their account, email data, and activity logs from the system.
Data Security: All email content and metadata are encrypted for storage and transmitted securely using TLS.
Minimal Retention: Only essential information (e.g., classification results, timestamps) is retained; raw email content can be deleted after processing.

5.2 Ethical Considerations

Not for Misuse: The system is designed solely to detect spam and improve email security, not for monitoring or accessing private communications without consent.

Bias Mitigation: Classification models are trained on diverse email patterns to reduce bias and ensure fair detection across different senders and content types.

Transparency: Users are informed about spam likelihood or confidence scores to avoid misinterpretation of results..

Responsible Use: The system is intended for improving email security and productivity, not for unauthorized surveillance or data exploitation.

Chapter 6
CONCLUSION AND FUTURE WORK CONCLUSION

Conclusion

The Spam Email Detection system demonstrates how AI and machine learning can be applied to enhance email security and productivity. By analyzing email content, metadata, and sender information, the system effectively identifies and filters spam messages, reducing user risk and improving workflow efficiency. The project emphasizes privacy, consent, and transparency, ensuring that sensitive email data is handled responsibly and ethically.

Through the system's architecture and testing, the importance of robust preprocessing, diverse datasets, and clear confidence scores is evident for maintaining accurate detection and user trust. While initial results show promising performance, further validation and optimization are needed to ensure reliability across different email types, languages, and spam patterns.

This project establishes a strong foundation for a scalable spam detection platform, balancing technological effectiveness with ethical responsibility and preparing the system for future improvements.

Future Work

- Improve classification accuracy across diverse email formats, languages, and spam techniques.
- Expand and diversify the dataset to reduce model bias and enhance generalization.
- Provide more detailed feedback to users on why an email is classified as spam.
- Integrate multi-layered detection, including phishing, malware links, and attachments.

Chapter 7

REFERENCES

REFERENCES

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
2. Joulin, A., Grave, E., Bojanowski, P., Mikolov, T. (2017). Bag of Tricks for Efficient Text Classification. *arXiv preprint arXiv:1607.01759*.
3. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research, 16*, 321–357.
4. Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.
5. Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
6. Microsoft Azure. (2023). Text Analytics API Documentation. https://learn.microsoft.com/en-us/azure/cognitive-services/text-analytics/
7. European Union. (2016). General Data Protection Regulation (GDPR). https://gdpr.eu/
8. Government of India. (2023). Digital Personal Data Protection Act (DPDP).
9. King, D. E. (2009). Dlib-ml: A Machine Learning Toolkit. *Journal of Machine Learning Research, 10*, 1755–1758.
10. Zhang, Y., Wallace, B. (2015). A Sensitivity Analysis of (and Practitioners' Guide to) Convolutional Neural Networks for Sentence Classification. *arXiv preprint arXiv:1510.0*